Trabalho final Análise Numérica

Luiz Fernando Bossa* Prof. Fermín S. V. Bazán

Dezembro 2025

[ ]:
```
using Plots
using LinearAlgebra
```

1 Trabalho final Análise Numérica

1.1 Considere o problema

$$\begin{cases} u_{tt} - u_{xx} = 0 & x \in [0,\pi], t \geq 0 \\ u(x,0) = \exp(-100(x-\pi/2)^2), \\ u_t(x,0) = 0 \\ u(0,t) = 0, \\ u_x(\pi,t) + \delta u_t(\pi,t) = 0, \end{cases}$$

Descreva métodos numéricos para resolver o problema de ondas 1D, incluindo a análise de estabilidade dos esquemas discretos. Considere $\delta \in (0,1]$. Apresente resultados numéricos para valores distintos de t que ilustrem a evolução da solução.

1.1.1 Preparando a EDP

Vamos denotar por

$$U_{i,j} = u(x_i, t_j)$$

e fazemos

$$x_i = ih, \qquad i = 0, \ldots, N \tag{1}$$

$$t_j = jk, \qquad j = 0, \ldots, T \tag{2}$$

[ ]:
```
f(x) = exp(-100(x-π/2)^2) # curva inicial
```

[ ]:
```
f (generic function with 1 method)
```

[ ]:
```
# Vamos definir o domínio e plotar a curva inicial
N = 314*2
h = π/(N-1)
domain = 0:h:π;
h
```

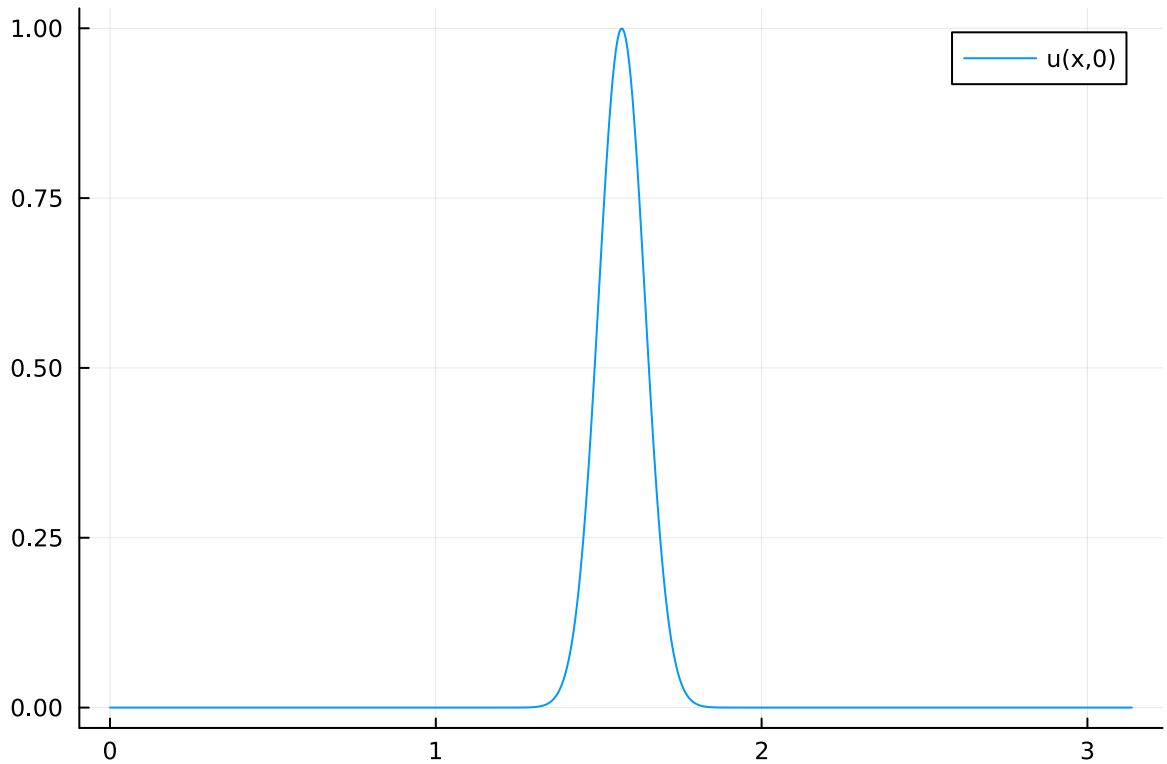
[ ]:
```
0.005010514599026784
```

*l.f.bossa@posgrad.ufsc.br

Vamos usar $h \approx$ `5e-3`. Vejamos o perfil inicial da nossa onda.

[ ]:
```
plot(domain, f.(domain), label="u(x,0)")
```

[ ]:



Vamos analisar suas condições de fronteira e como elas se traduzem em relação aos $U_{i,j}$.

- $u_t(x, 0) = 0 $ nos diz que a velocidade inicial é zero.

Usando uma aproximação de primeira ordem, podemos escrever:

$$\frac{\partial}{\partial t}u(x,0) \approx \frac{U_{i,1} - U_{i,0}}{k} = 0 \implies U_{i,1} = U_{i,0}$$

- $u(0, t) = 0 $ **Condição de Diriclet**, nos diz que a onda está fixa em $(0,0)$, ou seja, $U_{0,j} \equiv 0$
- $u_x(\pi, t) + \delta u_t(\pi, t) = 0$ **Condição de Robin/Impedância**: nos diz que na ponta $x = \pi$, parte da onda é refletida e parte é absorvida.

Usando uma aproximação de primeira ordem, podemos escrever

$$\frac{U_{N,j} - U_{N-1,j}}{h} + \delta \frac{U_{N,j} - U_{N,j-1}}{k} = 0$$

e fazendo $\lambda = \frac{k}{h}$ podemos isolar $U_{N,j}$

$$U_{N,j} = \frac{\delta}{\lambda + \delta} U_{N,j-1} + \frac{\lambda}{\lambda + \delta} U_{N-1,j}$$

1.1.2 Leapfrog

Esquema da molécula do Leapfrog.

Usando uma aproximação de segunda ordem para u_{xx} e u_{tt} temos

$$\frac{U_{i+i,j} - 2U_{i,j} + U_{i-1,j}}{h^2} = \frac{U_{i,j+1} - 2U_{i,j} + U_{i,j-1}}{k^2}$$

Isolando $U_{i+1,j}$ temos

$$U_{i,j+1} = 2U_{i,j} - U_{i,j-1} + \lambda^2 \left(U_{i+1,j} - 2U_{i,j} + U_{i-1,j}\right), \qquad i = 1, \ldots, N-1$$

para $i = 0$ e $i = N$, usamos as condições de fronteira descritas anteriormente.

1.1.3 Análise von Neumann + CFL

Fazendo

$$U_{i,j} = \xi^i e^{\infty j\theta}$$

com $\infty = \sqrt{-1}$

Para que o método seja estável, o erro não pode crescer com o tempo, logo precisamos que $|\xi| \leq 1$

Substituímos o ansatz na equação de diferenças

$$\xi^{i+1} e^{\infty j\theta} = 2\xi^i e^{\infty j\theta} - \xi^{i-1} e^{\infty j\theta} + \lambda^2 \left[\xi^i e^{\infty (j+1)\theta} - 2\xi^i e^{\infty j\theta} + \xi^i e^{\infty (j-1)\theta}\right]$$

Dividimos toda a equação por $\xi^{i-1} e^{\infty j\theta}$

$$\xi^2 = 2\xi - 1 + \lambda^2 \xi \left[e^{\infty\theta} - 2 + e^{-\infty\theta}\right]$$

Usamos a identidade de Euler $e^{\infty\theta} + e^{-\infty\theta} = 2\cos\theta$

$$\xi^2 - 2\xi + 1 = \lambda^2 \xi (2\cos\theta - 2)$$

Reorganizando o termo da direita usando a identidade trigonométrica $1 - \cos\theta = 2\sin^2\left(\frac{\theta}{2}\right)$

$$2\cos\theta - 2 = -2(1 - \cos\theta) = -4\sin^2\left(\frac{\theta}{2}\right)$$

Substituindo de volta na equação

$$\xi^2 - 2\xi + 1 = \xi \left[-4\lambda^2 \sin^2\left(\frac{\theta}{2}\right)\right]$$

Isso nos dá a equação característica quadrática

$$\xi^2 - 2\left[1 - 2\lambda^2 \sin^2\left(\frac{\theta}{2}\right)\right]\xi + 1 = 0$$

Para simplificar, vamos definir $A = 1 - 2\lambda^2 \sin^2(\frac{\theta}{2})$.

A equação fica

$$\xi^2 - 2A\xi + 1 = 0$$

As raízes desta equação quadrática são:

$$\xi = \frac{2A \pm \sqrt{4A^2 - 4}}{2} = A \pm \sqrt{A^2 - 1}$$

Para que o método seja estável ($|\xi| \leq 1$), devemos analisar o termo dentro da raiz (discriminante):

- Se $A^2 > 1$ (Raízes Reais): As raízes são reais e distintas. Como o produto das raízes é $c/a = 1$, se elas são reais e não são ambas 1 ou -1, uma delas terá magnitude maior que 1. Isso resultaria em instabilidade.
- Se $A^2 \leq 1$ (Raízes Complexas): O discriminante é negativo ou zero. As raízes são $A \pm \infty\sqrt{1 - A^2}$. O módulo ao quadrado

  $$|\xi|^2 = A^2 + (\sqrt{1 - A^2})^2 = A^2 + 1 - A^2 = 1$$

  Neste caso, $|\xi| = 1$. Isso indica estabilidade neutra (o erro não cresce nem diminui, apenas oscila, o que é ideal para a equação da onda pois não introduz dissipação artificial).Portanto, a condição de estabilidade é

$$|A| \leq 1$$

$$-1 \leq 1 - 2\lambda^2 \sin^2\left(\frac{\theta}{2}\right) \leq 1$$

$$0 \leq 2 - 2\lambda^2 \sin^2\left(\frac{\theta}{2}\right) \leq 2$$

$$0 \leq 1 - \lambda^2 \sin^2\left(\frac{\theta}{2}\right) \leq 1$$

A desigualdade do lado direito é claramente satisfeita. A do lado esquerdo precisa que

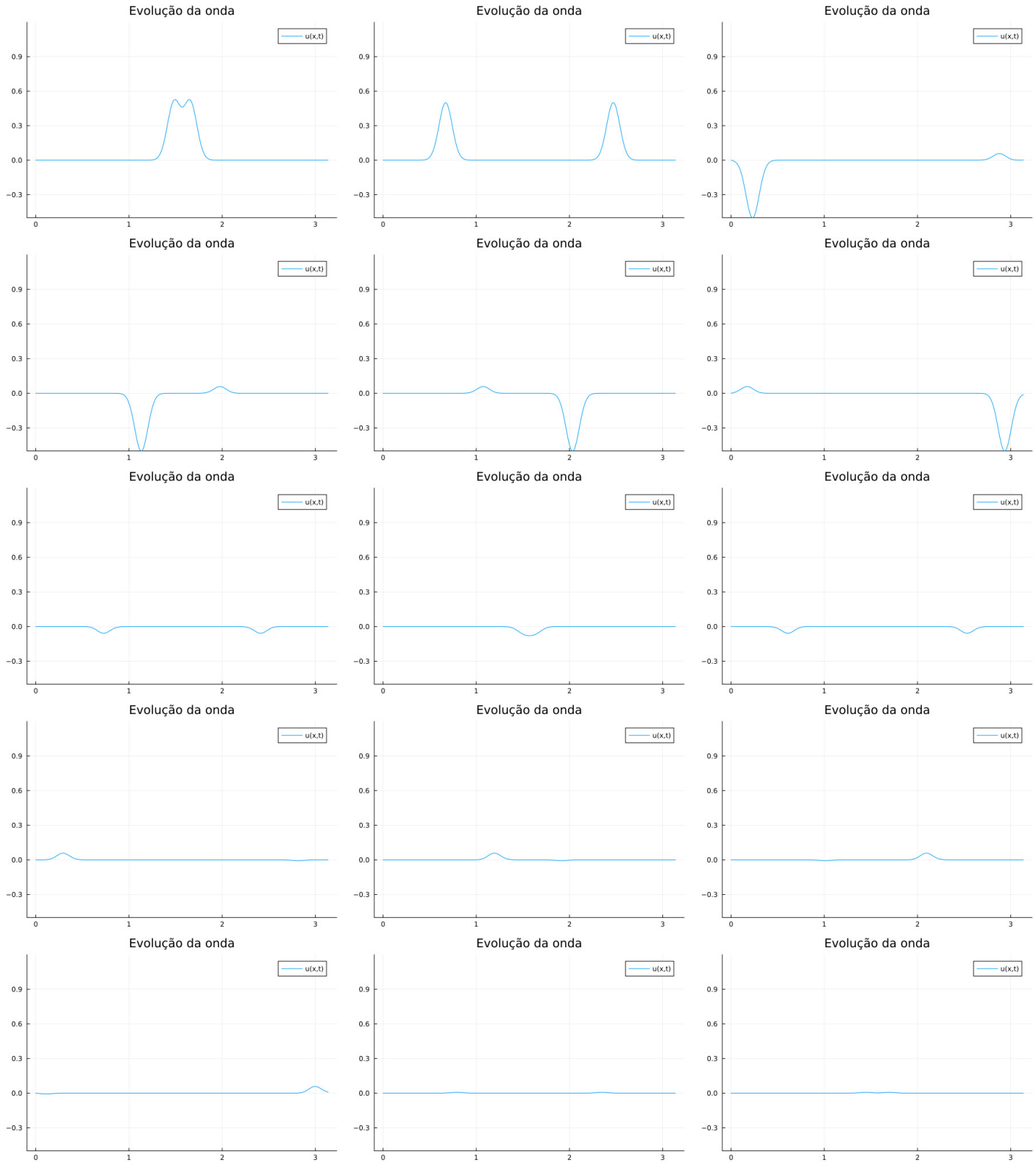
$$\lambda^2 \sin^2\left(\frac{\theta}{2}\right) \leq 1$$

o que significa que para garantir a condição Courant-Friedrichs-Lewy devemos tomar $|\lambda| \leq 1$.

1.1.4 Resolvendo

[ ]:
```
# LeapFrog
δ = 0.8 # parâmetro de absorção
λ = 0.9 #
k = λ*h
```

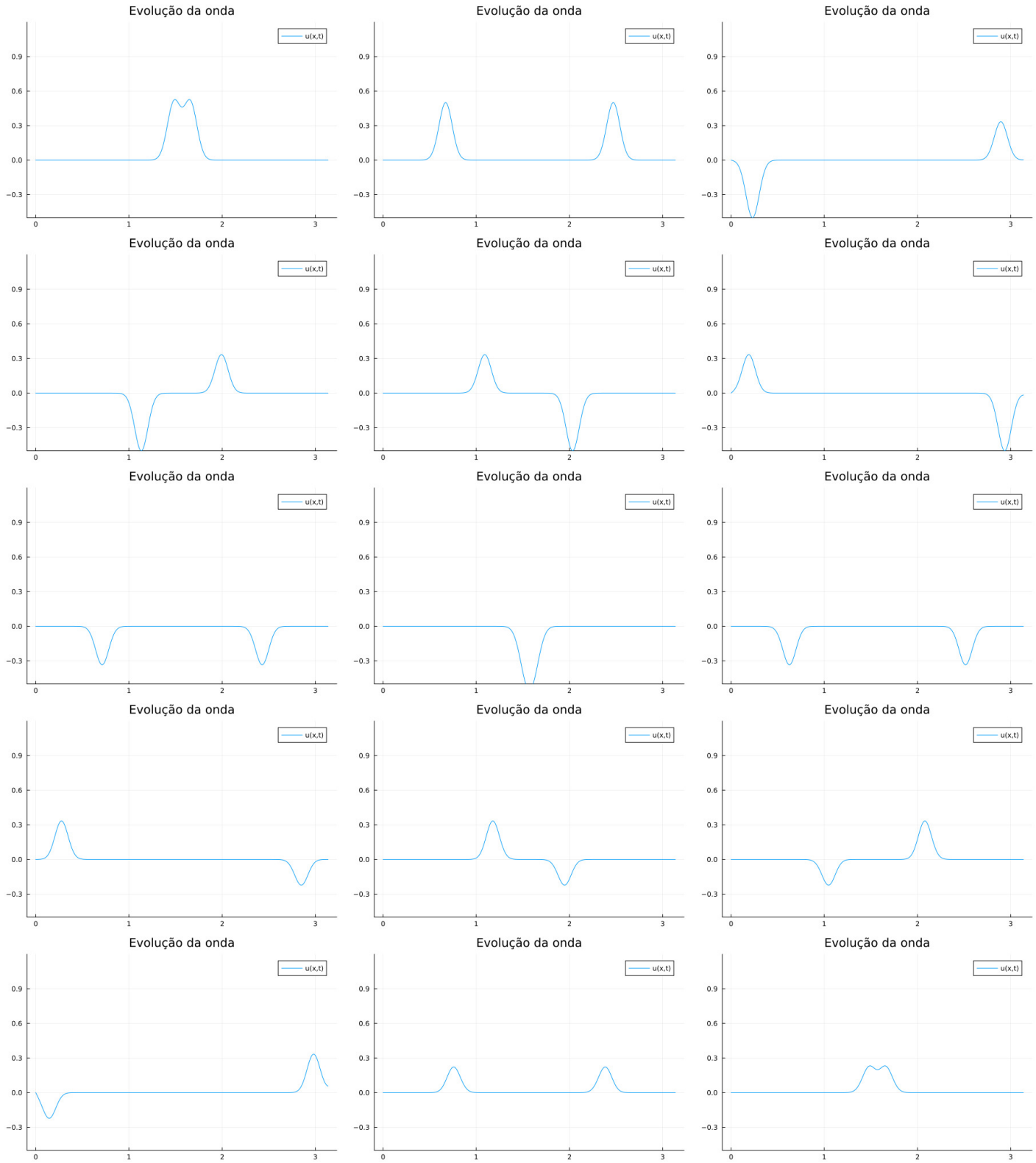
```
T = 5000

U0 =  f.(domain);
U1 = copy(U0); # U^1 = U^0
U = [U0, U1]
M = length(U0)

for j=2:T
  Ujp1 = zeros(M) # Ut[0] = 0 por condjção jnjcjal
  Uj = U[j]
  Ujm1 = U[j-1]
  for i=2:M-1  # para os pontos jnterjores fazemos o esquema leapfrog
    Ujp1[i] = 2Uj[i] - Ujm1[i] + λ^2*(Uj[i+1]+Uj[i-1] - 2*Uj[i])
  end
  # e para o ponto da frontejra, usamos uma aproxjmação de 1a ordem
  Ujp1[M] = δ/(λ+δ)*Uj[M] + λ/(λ+δ)*Ujp1[M-1]
  push!(U, Ujp1)
end
```



[ ]:
```
# LeapFrog
δ = 0.2 # parâmetro de absorção
λ = 0.9 #
k = λ*h
T = 5000
```

```
U0 =  f.(domain);
U1 = copy(U0); # U^1 = U^0
U = [U0, U1]
M = length(U0)

for j=2:T
  Ujp1 = zeros(M) # Ut[0] = 0 por condjção jnjcjal
  Uj = U[j]
  Ujm1 = U[j-1]
  for i=2:M-1  # para os pontos jnterjores fazemos o esquema leapfrog
    Ujp1[i] = 2Uj[i] - Ujm1[i] + λ^2*(Uj[i+1]+Uj[i-1] - 2*Uj[i])
  end
  # e para o ponto da frontejra, usamos uma aproxjmação de 1a ordem
  Ujp1[M] = δ/(λ+δ)*Uj[M] + λ/(λ+δ)*Ujp1[M-1]
  push!(U, Ujp1)
end
```

[ ]:
```
@gif for t=1:20:T
  plot(domain, U[t], ylim=(-0.5,1.2), label="u(x,t)", title="Evolução da onda")
end fps=60
```



Veja que o parâmetro $\delta = 0.8$ causou um efeito maior de absorção, enquanto um $\delta = 0.2$ causou um efeito maior de reflexão.

1.1.5 Método Box

A equação da onda:

$$u_{tt} - u_{xx} = 0$$

Defina:

$$v = u_t \quad \text{e} \quad w = u_x$$

Então temos o sistema:

$$\begin{cases} v_t - w_x = 0 \\ w_t - v_x = 0 \end{cases}$$

1.1.6 Condições Iniciais e de Contorno

Condições iniciais:

$$u(x,0) = \exp(-100(x - \pi/2)^2)$$

$$v(x,0) = u_t(x,0) = 0$$

$$w(x,0) = u_x(x,0) = -200(x - \pi/2)\exp(-100(x - \pi/2)^2)$$

Condições de contorno:

$$u(0,t) = 0 \Rightarrow v(0,t) = 0$$

$$u_x(\pi,t) + \delta u_t(\pi,t) = 0 \Rightarrow w(\pi,t) + \delta v(\pi,t) = 0$$

1.1.7 Discretização com Método Box

Em cada direção, calculamos a média das aproximações por diferenças progressivas.

Equação 1 $v_t - w_x = 0$**:**

$$\frac{V_{i+1,j+1} - V_{i+1,j} + V_{i,j+1} - V_{i,j}}{2k} - \frac{W_{i+1,j+1} - W_{i,j+1} + W_{i+1,j} - W_{i,j}}{2h} = 0$$

Equação 2 $w_t - v_x = 0$**:**

$$\frac{W_{i+1,j+1} - W_{i+1,j} + W_{i,j+1} - W_{i,j}}{2k} - \frac{V_{i+1,j+1} - V_{i,j+1} + V_{i+1,j} - V_{i,j}}{2h} = 0$$

Onde: - $ V_{i,j} = v(x_i, t_j) $, - $ W_{i,j} = w(x_i, t_j) $,

1.1.8 Sistema Linear para Cada Passo de Tempo

Para cada passo no tempo, montamos um sistema linear

$$A\mathbf{U}^{j+1} = \mathbf{b},$$

onde:

$$\mathbf{U}^{j+1} = [v_0, w_0, v_1, w_1, \ldots, v_N, w_N]^T$$

[ ]:
```
# Box
δ = 0.8 # parâmetro de absorção
λ = 0.9 #
k = λ*h
```

```
T = 4000

U0 =  f.(domain);
U1 = copy(U0); # U^1 = U^0
U = [U0, U1]
M = length(U0)


function w(x) #u_x
    return -200*(x - π/2) * exp(-100*(x - π/2)^2)
end

V = [zeros(M)]
W = [w.(domain)]

ν = k / h


# Método Box para a equação da onda

for j=1:T

    Vj = V[j]
    Wj = W[j]
    Uj = U[j+1]
    # Montar sistema linear A * U_{j+1} = b

    # Número de equações:
    # 2 equações para cada intervalo
    # + 2 condições de contorno -> total 2(M) = 2(N+1)
    n_equations = 2*M
    A = spzeros(n_equations, n_equations)
    b = zeros(n_equations)

    eq_counter = 0

    # Condição de contorno em x=0: v(0,t)=0
    eq_counter += 1
    A[eq_counter, 1] = 1.0  # v_0
    b[eq_counter] = 0.0

    # Equações para cada intervalo interno (i=1:N)
    for i in 1:M-1
        # Equação 1 para o intervalo (i, i+1)
        eq_counter += 1
        # Coeficientes para v_i, v_{i+1}, w_i, w_{i+1}
        v_i_idx = 2*(i-1) + 1
        w_i_idx = 2*(i-1) + 2
        v_ip1_idx = 2*i + 1
```

```
        w_ip1_idx = 2*i + 2

        # v_i + v_{i+1} + ν*w_i - ν*w_{i+1} = R_v
        A[eq_counter, v_i_idx] = 1.0
        A[eq_counter, v_ip1_idx] = 1.0
        A[eq_counter, w_i_idx] = ν
        A[eq_counter, w_ip1_idx] = -ν

        # Lado direito R_v
        R_v = Vj[i] + Vj[i+1] + ν*(Wj[i+1] - Wj[i])
        b[eq_counter] = R_v

        # Equação 2 para o intervalo (i, i+1)
        eq_counter += 1
        # w_i + w_{i+1} + ν*v_i - ν*v_{i+1} = R_w
        A[eq_counter, w_i_idx] = 1.0
        A[eq_counter, w_ip1_idx] = 1.0
        A[eq_counter, v_i_idx] = ν
        A[eq_counter, v_ip1_idx] = -ν

        # Lado direito R_w
        R_w = Wj[i] + Wj[i+1] + ν*(Vj[i+1] - Vj[i])
        b[eq_counter] = R_w
    end

    # Condição de contorno em x=π: w_N + δ*v_N = 0
    eq_counter += 1
    v_N_idx = 2*(M-1) + 1
    w_N_idx = 2*(M-1) + 2
    A[eq_counter, v_N_idx] = δ
    A[eq_counter, w_N_idx] = 1.0
    b[eq_counter] = 0.0

    # Resolver sistema linear
    U_next = A \ b

    # Extrair soluções
    Vjp1 = zeros(M)
    Wjp1 = zeros(M)
    for i in 1:M
        Vjp1[i] = U_next[2*(i-1)+1]
        Wjp1[i] = U_next[2*(i-1)+2]
    end

    push!(V, Vjp1)
    push!(W, Wjp1)

    # Atualizar u usando regra do trapézio
    Ujp1 = zeros(M)
    for i in 1:M
```

```
        Ujp1[i] = Uj[i] + (k/2)*(Vj[i] + Vjp1[i])
    end
    push!(U, Ujp1)
end
```

1.2 [5.38]

Resolver a equação:

$$u_t + a(x,t)u_x = 0, \quad x \geq 0, \quad t \geq 0,$$

onde

$$a(x,t) = \frac{1+x^2}{1+2xt+2x^2+x^4},$$

com a condição inicial:

a.

$$u(x,0) = \begin{cases} 1 & 0.2 \leq x \leq 0.4 \\ 0 & \text{caso contrário} \end{cases} \tag{3}$$

$$u(0,t) = 0, \tag{4}$$

cuja solução exata é $u(x,t) = u\left(\frac{x-t}{1+x^2}, 0\right)$.

b.

$$u(x,0) = \exp(-10(4x-1)^2) \tag{5}$$

$$u(0,t) = 0. \tag{6}$$

Usar malhas com $h = 0.02$ e $\Delta t = 0.01$.

Compare os perfis das soluções em $t = 0$, $t = 0.1$, $t = 0.5$ e $t = 1$, obtidas pelos métodos - **upwind**, - **Lax-Wendroff**, - **método Box** - **esquema implícito de primeira ordem**.

1.2.1 (a) Condição inicial quadrada

[ ]:
```
a(x,t) = (1+x^2)/(1+2x*t + 2x^2 + x^4)

u(x) = begin
  if 0.2 <= x <= 0.4
    return 1.0
  else
    return 0.0
  end
end
```

[ ]:
```
u (generic function with 1 method)
```

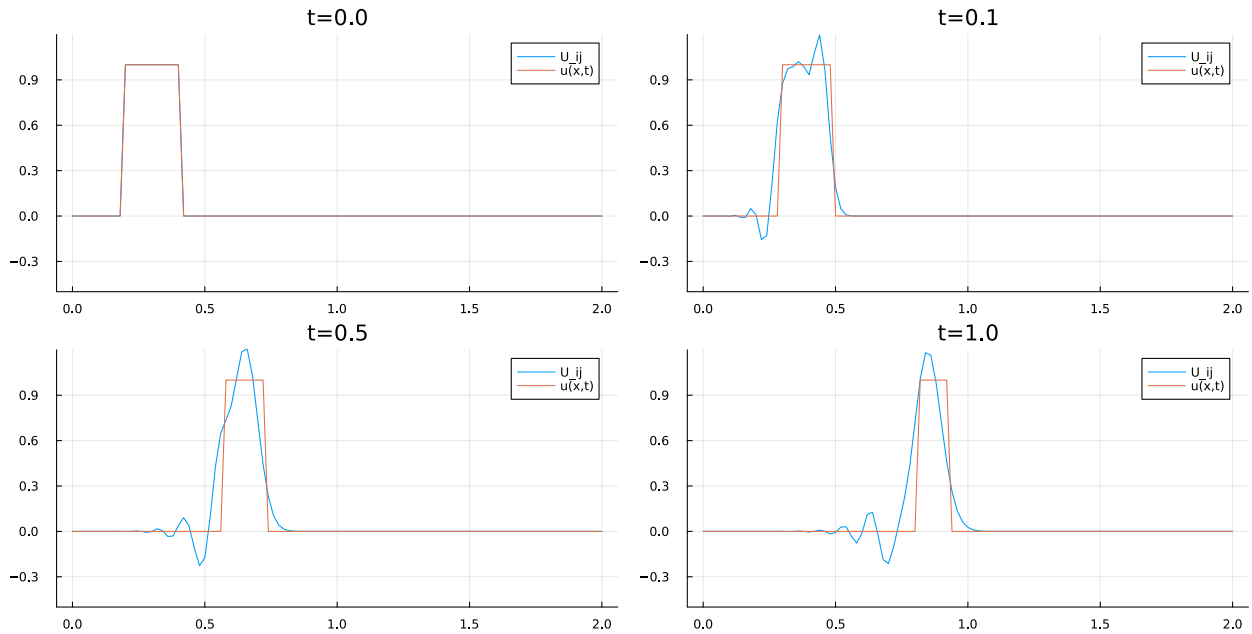
Lax-Wendroff

$$U_{i,j+1} = U_{i,j} - \frac{\nu a}{2}(U_{i+1,j} - U_{i-1,j}) + \frac{\nu^2 a^2}{2}(U_{i+1,j} - 2U_{i,j} + U_{i-1,j})$$

[ ]:
```
# Lax-Wendroff
h = 0.02
k = 0.01

T = 100

domain = 0:h:2

U0 =  u.(domain);

ν = k/h

U = [U0]


M = length(U0)

for j=1:T
  Ujp1 = zeros(M) # Ut[0] = 0 por condição inicial
  Uj = U[j]
  for i=2:M-1  # para os pontos jnterjores fazemos o esquema Lax-Wendroff
    xi = i*h
    tj = j*k
    va = ν*a(xi,tj)
    Ujp1[i] = Uj[i] - va/2*(Uj[i+1]-Uj[i-1]) +
                      va^2/2*(Uj[i+1] -2Uj[i] + Uj[i-1])
  end
  push!(U, Ujp1)
end
```

[ ]:
```
graficos = []
for t=[1,11,51,101 ]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.2), label="U_ij", title="t=$tt")
  solu(z) = z- tt/(1+z^2)
  plot!(p, domain, u.(solu.(domain)), label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Observamos uma suavização ao longo do tempo, uns spikes de erros numéricos, e um efeito de aumento de amplitude da onda.

Esquema implícito de primeira ordem

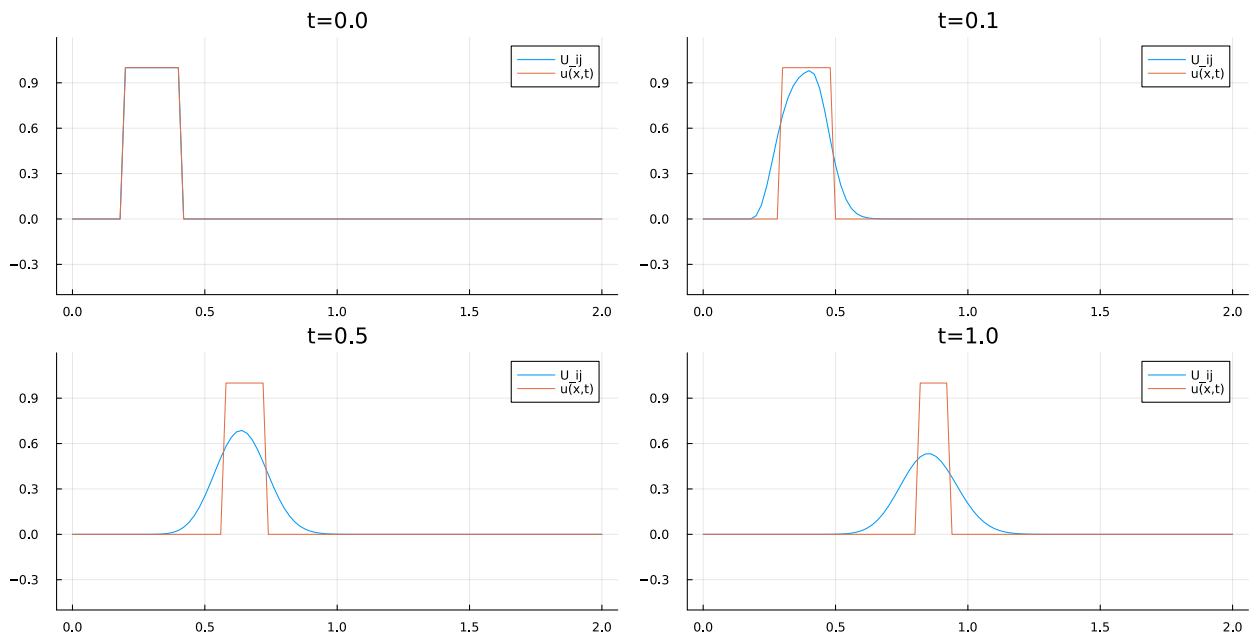
$$U_{i,j} = (1 + \nu a)U_{i,j+1} - \nu a U_{i-1,j+1}$$

[ ]:
```
# Implicito Primeira Ordem
h = 0.02
k = 0.01

T = 100

domain = 0:h:2

U0 =  u.(domain);


U = [U0]
ν = k/h

M = length(U0)

for j=1:T
  Uj = U[j]
  diag = ones(M)
  subdiag = zeros(M-1)
  for i=1:M-1
    xi = (i+1)*h
    tj = j*k
```

```
    va = ν*a(xi,tj)
    diag[i+1] = 1 + va
    subdiag[i] = -va
  end
  A = Tridiagonal(subdiag, diag, zeros(M-1))
  Ujp1 = A\Uj
  push!(U, Ujp1)
end
```

[ ]:
```
graficos = []
for t=[1,11,51,101 ]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.2), label="U_ij", title="t=$tt")
  solu(z) = z- tt/(1+z^2)
  plot!(p, domain, u.(solu.(domain)), label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Temos uma suavização da descontinuidade bem como um efeito de dissipação.

Método Box

$$U_{i+1,j+1} + \frac{1-\nu a}{1+\nu a} U_{i,j+1} = U_{i,j} + \frac{1-\nu a}{1+\nu a} U_{i+1,j}$$

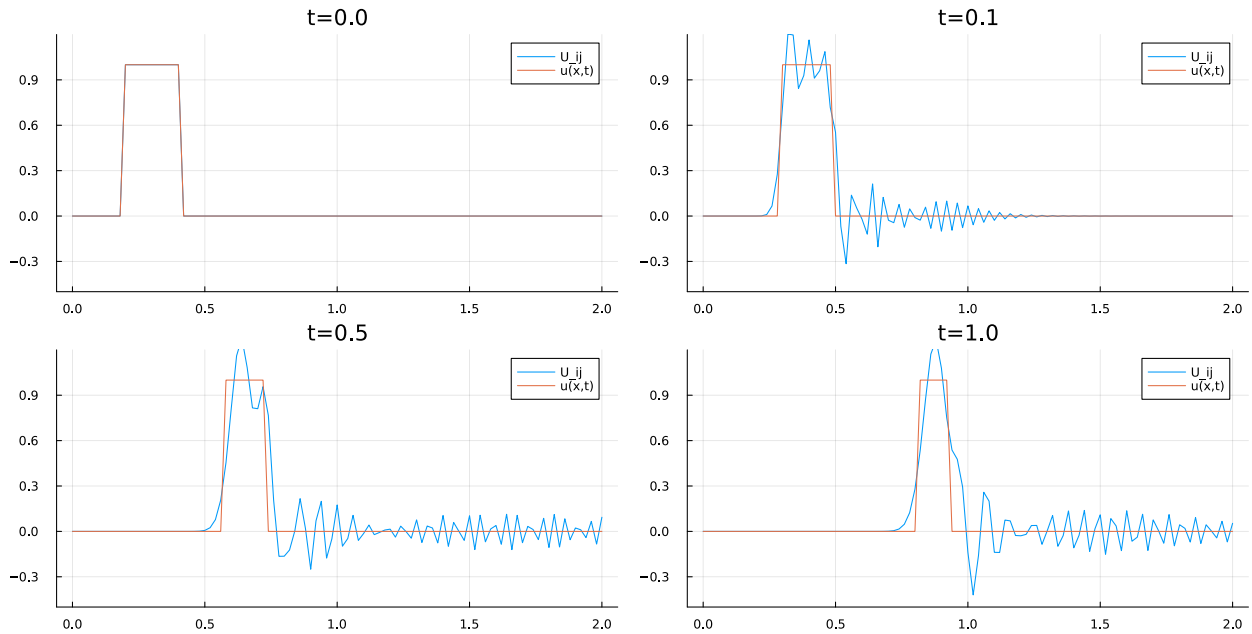
[ ]:
```
 # Método Box
h = 0.02
```

```
k = 0.01

T = 100

domain = 0:h:2

U0 =  u.(domain);


U = [U0]
ν = k/h

M = length(U0)


for j=1:T
  Uj = U[j]

  Ujp1 = zeros(M)
  diagA = ones(M)
  subdiagB = zeros(M)
  for i=1:M-1
    xi = (i-1)*h  # Correção: índice começa em 0
    tj = (j-1)*k
    va = ν*a(xi,tj)
    λ = (1 - va)/(1 + va)
    if i == 1
    # Para i=1: U_{1,j+1} = 0 (condição de contorno)
    # Equação: U_{2,j+1} + λ * 0 = U_{1,j} + λ * U_{2,j}
        Ujp1[2] = Uj[1] + λ * Uj[2]
    else
        # Para i>1: U_{i+1,j+1} + λ * U_{i,j+1} = U_{i,j} + λ * U_{i+1,j}
        # Isolando U_{i+1,j+1}:
        Ujp1[i+1] = Uj[i] + λ * Uj[i+1] - λ * Ujp1[i]
    end
  end
  push!(U,Ujp1)
end
```

[ ]:

```
graficos = []
for t=[1,11,51,101 ]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.2), label="U_ij", title="t=$tt")
  solu(z) = z- tt/(1+z^2)
  plot!(p, domain, u.(solu.(domain)), label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Método *upwind*

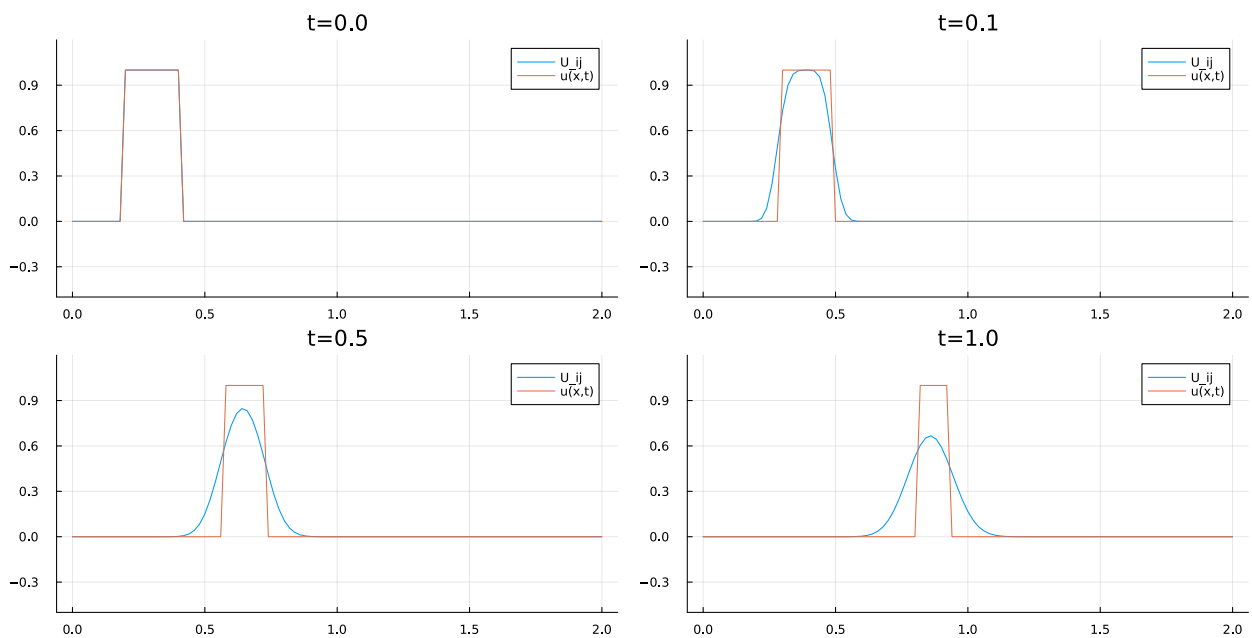
$$U_{i,j+1} = U_{i,j} - \frac{a\nu}{2}(U_{i+1,j} - U_{i-1,j}) + \frac{|a|\nu}{2}(U_{i+1,j} - 2U_{i,j} + U_{i-1,j})$$

[ ]:
```
# upwind
h = 0.02
k = 0.01

T = 100

domain = 0:h:2
U0 =  u.(domain);
U = [U0]
ν = k/h

M = length(U0)

for j=1:T
  Uj = U[j]
  Ujp1 = zeros(M)

  subdA = zeros(M-1)
  diagA = ones(M)
  supdA = zeros(M-1)

  subdB = zeros(M-1)
```

```
  diagB = ones(M)
  supdB = zeros(M-1)
  for i=2:M-1
    xi = i*h
    tj = j*k
    va = ν*a(xi,tj)
    vap = ν*abs(a(xi,tj))
    Ujp1[i] = Uj[i] - va/2*(Uj[i+1] - Uj[i-1]) + vap/2*(Uj[i+1] - 2*Uj[i] + Uj[i-1])
  end
  push!(U, Ujp1)
end
```

[ ]:
```
graficos = []
for t=[1,11,51,101 ]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.2), label="U_ij", title="t=$tt")
  solu(z) = z- tt/(1+z^2)
  plot!(p, domain, u.(solu.(domain)), label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Novamente temos um efeito de suavização e dissipação.

1.2.2 (b) Condição inicial suave

[ ]:
```
u(x) = exp(-10(4x - 1)^2)
```

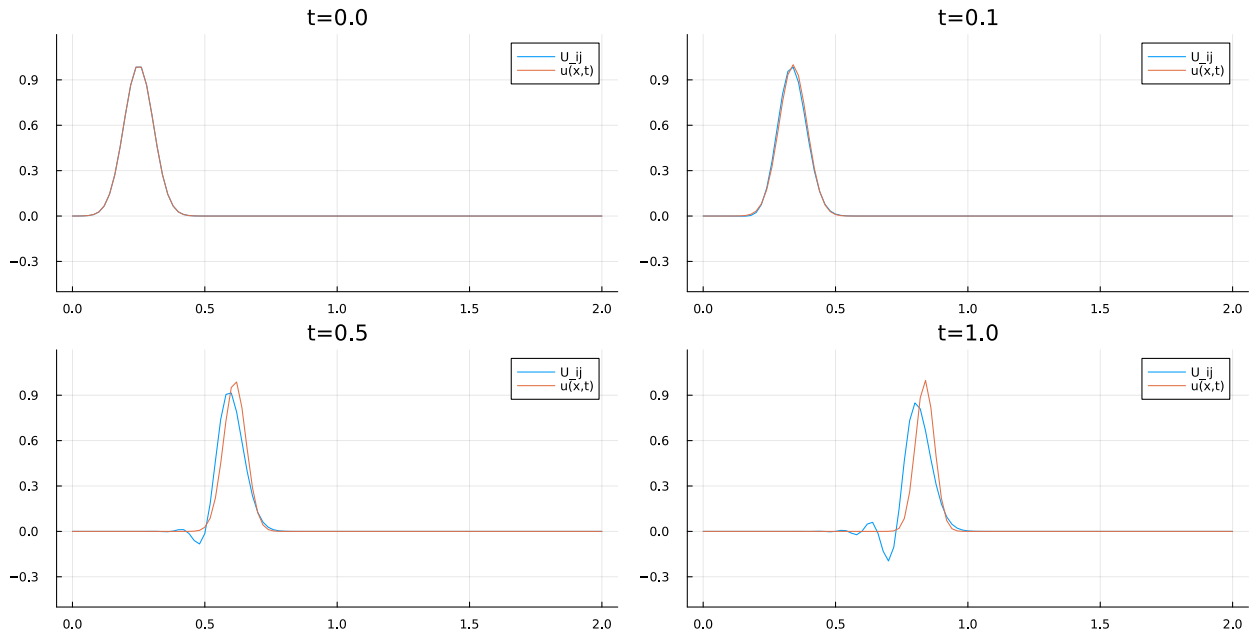
[ ]: u (generic function with 1 method)

Lax-Wendroff

[ ]:
```
# Lax-Wendroff
h = 0.02
k = 0.01

T = 100

domain = 0:h:2

U0 =  u.(domain);

ν = k/h

U = [U0]


M = length(U0)

for j=1:T
  Ujp1 = zeros(M) # Ut[0] = 0 por condição inicial
  Uj = U[j]
  for i=2:M-1  # para os pontos jnterjores fazemos o esquema Lax-Wendroff
    xi = i*h
    tj = j*k
    va = ν*a(xi,tj)
    Ujp1[i] = Uj[i] - va/2*(Uj[i+1]-Uj[i-1]) +
                      va^2/2*(Uj[i+1] -2Uj[i] + Uj[i-1])
  end
  push!(U, Ujp1)
end
```

[ ]:
```
graficos = []
for t=[1,11,51,101 ]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.2), label="U_ij", title="t=$tt")
  solu(z) = z- tt/(1+z^2)
  plot!(p, domain, u.(solu.(domain)), label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Esquema implícito de primeira ordem

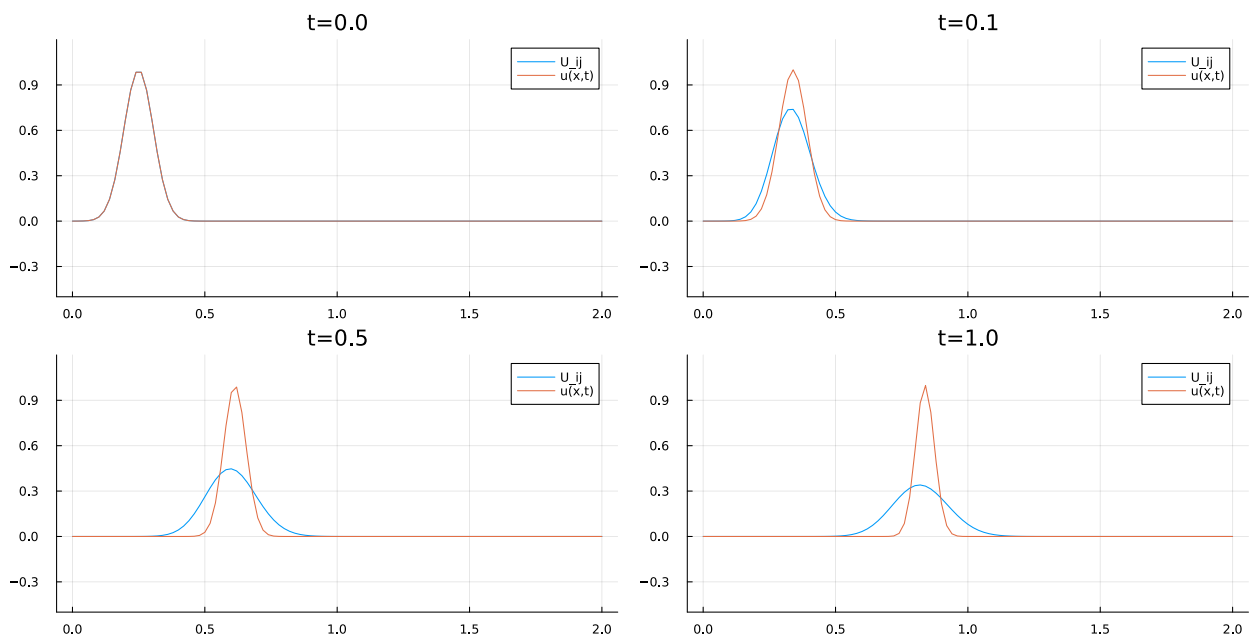
$$U_{i,j} = (1+\nu a)U_{i,j+1} - \nu a U_{i-1,j+1}$$

[ ]:
```
# Implicito Primeira Ordem
h = 0.02
k = 0.01

T = 100

domain = 0:h:2

U0 =  u.(domain);


U = [U0]
ν = k/h

M = length(U0)

for j=1:T
  Uj = U[j]
  diag = ones(M)
  subdiag = zeros(M-1)
  for i=1:M-1
    xi = (i+1)*h
```

```
    tj = j*k
    va = ν*a(xi,tj)
    diag[i+1] = 1 + va
    subdiag[i] = -va
  end
  A = Tridiagonal(subdiag, diag, zeros(M-1))
  Ujp1 = A\Uj
  push!(U, Ujp1)
end
```

[ ]:
```
graficos = []
for t=[1,11,51,101 ]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.2), label="U_ij", title="t=$tt")
  solu(z) = z- tt/(1+z^2)
  plot!(p, domain, u.(solu.(domain)), label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Método Box

$$U_{i+1,j+1} + \frac{1-\nu a}{1+\nu a} U_{i,j+1} = U_{i,j} + \frac{1-\nu a}{1+\nu a} U_{i+1,j}$$

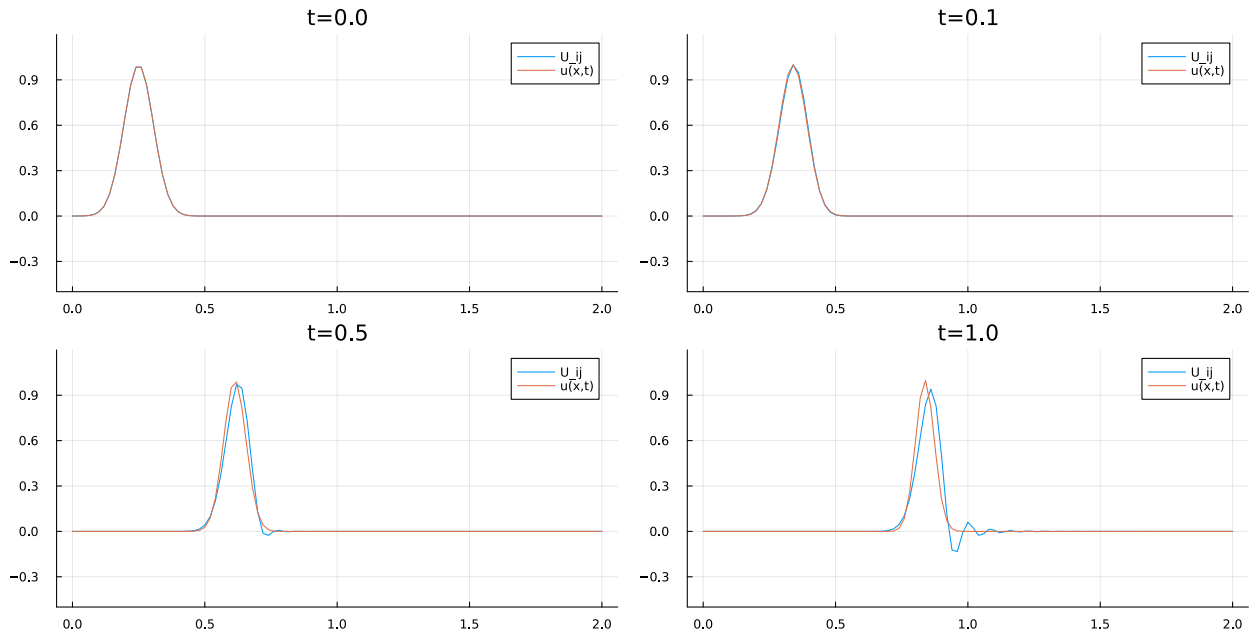
[ ]:
```
# Método Box
h = 0.02
k = 0.01
```

```
T = 100
domain = 0:h:2

U0 =  u.(domain);
U = [U0]
ν = k/h

M = length(U0)
for j=1:T
  Uj = U[j]

  Ujp1 = zeros(M)
  diagA = ones(M)
  subdiagB = zeros(M)
  for i=1:M-1
    xi = (i-1)*h  # Correção: índice começa em 0
    tj = (j-1)*k
    va = ν*a(xi,tj)
    λ = (1 - va)/(1 + va)
    if i == 1
    # Para i=1: U_{1,j+1} = 0 (condição de contorno)
    # Equação: U_{2,j+1} + λ * 0 = U_{1,j} + λ * U_{2,j}
        Ujp1[2] = Uj[1] + λ * Uj[2]
    else
        # Para i>1: U_{i+1,j+1} + λ * U_{i,j+1} = U_{i,j} + λ * U_{i+1,j}
        # Isolando U_{i+1,j+1}:
        Ujp1[i+1] = Uj[i] + λ * Uj[i+1] - λ * Ujp1[i]
    end
  end
  push!(U,Ujp1)
end
```

[ ]:

```
graficos = []
for t=[1,11,51,101 ]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.2), label="U_ij", title="t=$tt")
  solu(z) = z- tt/(1+z^2)
  plot!(p, domain, u.(solu.(domain)), label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Método *upwind*

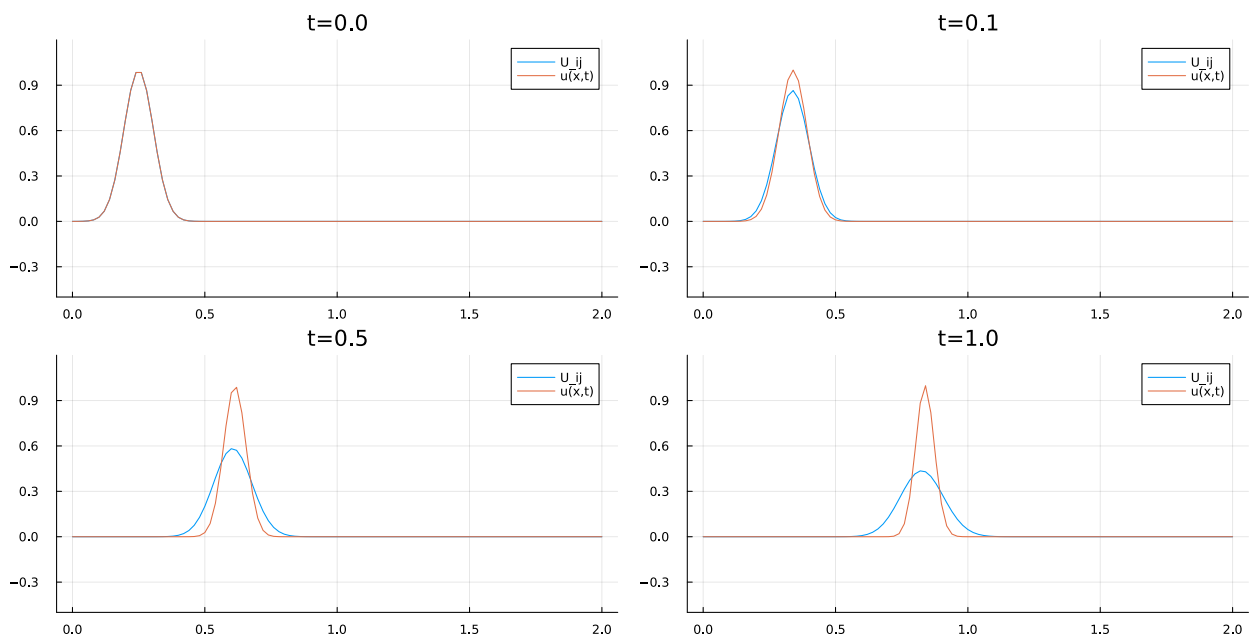
$$U_{i,j+1} = U_{i,j} - \frac{a\nu}{2}(U_{i+1,j} - U_{i-1,j}) + \frac{|a|\nu}{2}(U_{i+1,j} - 2U_{i,j} + U_{i-1,j})$$

[ ]:
```
# upwind
h = 0.02
k = 0.01

T = 100

domain = 0:h:2
U0 =  u.(domain);
U = [U0]
ν = k/h

M = length(U0)

for j=1:T
  Uj = U[j]
  Ujp1 = zeros(M)

  subdA = zeros(M-1)
  diagA = ones(M)
  supdA = zeros(M-1)

  subdB = zeros(M-1)
```

```
  diagB = ones(M)
  supdB = zeros(M-1)
  for i=2:M-1
    xi = i*h
    tj = j*k
    va = ν*a(xi,tj)
    vap = ν*abs(a(xi,tj))
    Ujp1[i] = Uj[i] - va/2*(Uj[i+1] - Uj[i-1]) + vap/2*(Uj[i+1] - 2*Uj[i] + Uj[i-1])
  end
  push!(U, Ujp1)
end
```

[ ]:
```
graficos = []
for t=[1,11,51,101 ]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.2), label="U_ij", title="t=$tt")
  solu(z) = z- tt/(1+z^2)
  plot!(p, domain, u.(solu.(domain)), label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



1.3 [5.39]

Resolver a equação de Burgers com as condições iniciais do problema anterior e mesmos valores para as malhas e comparação dos perfis. Use os métodos **Leap-Frog** e **Box**.

Considere a equação não-viscosa de Burgers

$$u_t + uu_x = 0$$

com condição inicial

$$u(x,0) = f(x)$$

A solução dessa equação é dada por

$$u(x,t) = f(x - ut)$$

Usar a aproximação de diferenças centradas.

$$\frac{U_{i,j+1} - U_{i,j-1}}{2k} + U_{i,j}\frac{U_{i+1,j} - U_{i-1,j}}{2h} = 0$$

$$U_{i,j+1} - U_{i,j-1} + \nu U_{i,j}(U_{i+1,j} - U_{i-1,j}) = 0$$

$$U_{i,j+1} = U_{i,j-1} - \nu U_{i,j}(U_{i+1,j} - U_{i-1,j})$$

Para conseguir inicializar o método, precisamos de $U_{i,1}$, que vamos calcular pelo método de Newton.

1.3.1 (a) Condição inicial quadrada

Observando as características, vemos a formação de uma onda de choque.

Essa onda de choque vai criar instabilidades numéricas.

Leapfrog

[ ]:
```
u(x) = begin
  if 0.2 <= x <= 0.4
    return 1.0
  else
    return 0.0
  end
end
```

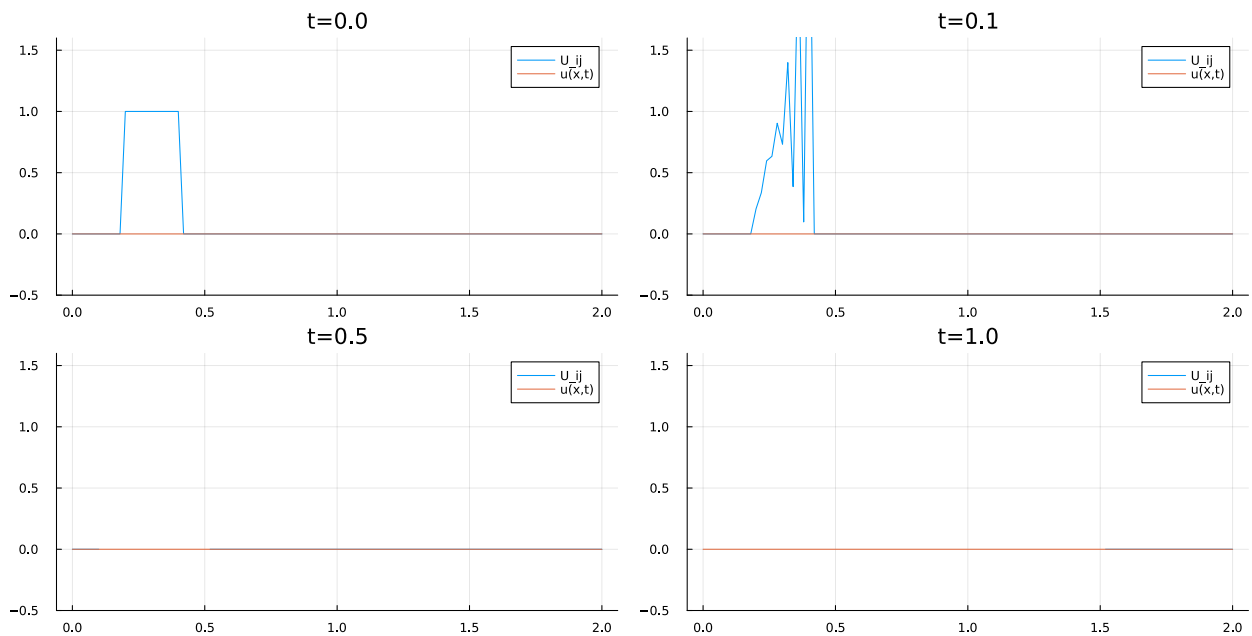
[ ]:
```
u (generic function with 1 method)
```

[ ]:
```
# Lax-Friedrich
h = 0.02
k = 0.01
T = 100
domain = 0:h:2
U0 =  u.(domain);
ν = k/h
M = length(U0)
U1 = zeros(M)
for i=2:M-1  # para os pontos interiores fazemos o esquema Lax-Friedrich
    U1[i] = U0[i] - ν/2*U0[i]*(U0[i+1] - U0[i-1])
end
```

```
U = [U0, U1]

for j=2:T
  Ujp1 = zeros(M) # Ut[0] = 0 por condição inicial
  Uj = U[j]
  Ujm1 = U[j-1]
  for i=2:M-1  # para os pontos interiores fazemos o esquema Lax-Wendroff
    Ujp1[i] = Ujm1[i] - ν*Uj[i]*(Uj[i+1] - Uj[i-1])
  end
  Ujp1[M] = Uj[M]
  push!(U, Ujp1)
end
```

[ ]:
```
graficos = []
for t=[1,11,51,101]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.6), label="U_ij", title="t=$tt")
  solucao = u.( domain .- u.(domain).*t )
  plot!(p, domain, solucao, label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Esse método claramente não consegue lidar com a onda de choque.

Método Box

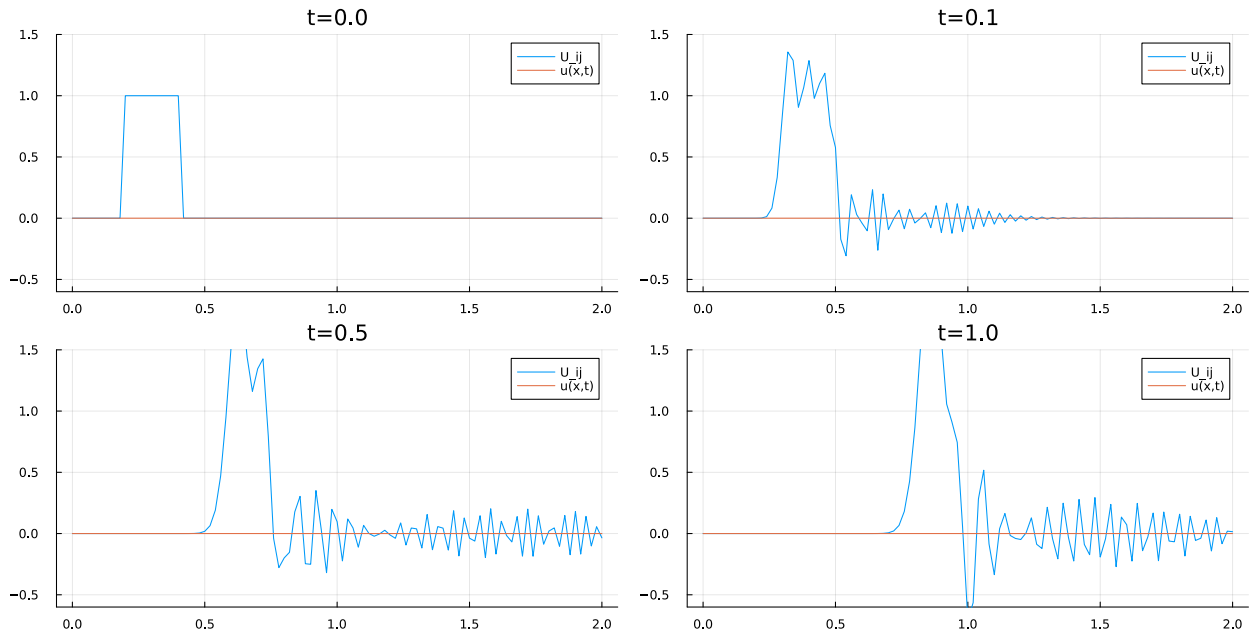
[ ]:
```
# Método Box
h = 0.02
k = 0.01
```

```
T = 100

domain = 0:h:2

U0 =  u.(domain);


U = [U0]
ν = k/h

M = length(U0)


for j=1:T
  Uj = U[j]
  diagA = ones(M)
  subdiagB = zeros(M)
  for i=2:M-1
    xi = i*h
    tj = j*k
    va = ν*a(xi,tj)
    λ = (1 + va)/(1 - va)
    diagA[i] = λ
    subdiagB[i] = λ
  end
  A = Tridiagonal(ones(M-1), diagA, zeros(M-1))
  B = Tridiagonal(subdiagB[1:M-1], ones(M), zeros(M-1))
  Ujp1 = A\(B*Uj)
  push!(U, Ujp1)
end
```

[ ]:

```
graficos = []
for t=[1,11,51,101]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.6,1.5), label="U_ij", title="t=$tt")
  solucao = u.( domain .- u.(domain).*t )
  plot!(p, domain, solucao, label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Esse método até consegue trabalhar mesmo com a onda de choque, mas gera essa sujeira na frente da onda.

1.3.2 (b) Condição inicial suave

Observando as características, vemos a formação de uma onda de choque.

Essa onda de choque é pior ainda, pois as características não são nem paralelas.

Leapfrog

[ ]:
```
u(x) = exp(-10(4x - 1)^2)
```

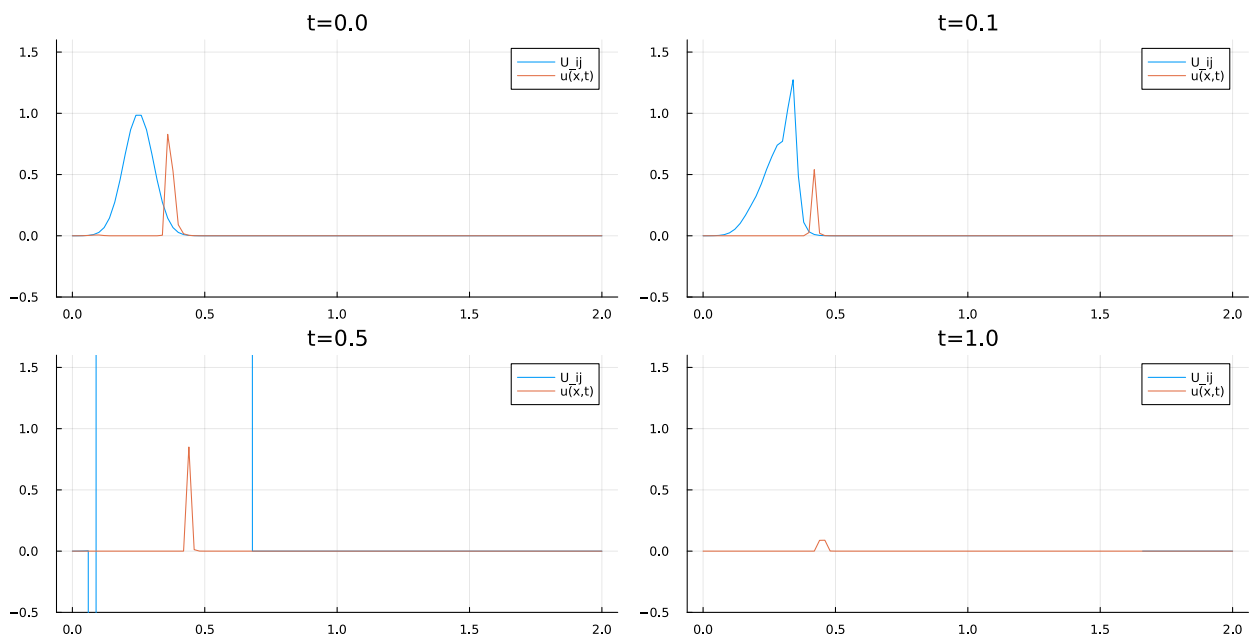
[ ]:
```
u (generic function with 1 method)
```

[ ]:
```
# Lax-Friedrich
h = 0.02
k = 0.01
T = 100
domain = 0:h:2
U0 =  u.(domain);
ν = k/h
M = length(U0)
U1 = zeros(M)
for i=2:M-1  # para os pontos interiores fazemos o esquema Lax-Friedrich
    U1[i] = U0[i] - ν/2*U0[i]*(U0[i+1] - U0[i-1])
end


U = [U0, U1]
```

```
for j=2:T
  Ujp1 = zeros(M) # Ut[0] = 0 por condição inicial
  Uj = U[j]
  Ujm1 = U[j-1]
  for i=2:M-1  # para os pontos interiores fazemos o esquema Lax-Wendroff
    Ujp1[i] = Ujm1[i] - ν*Uj[i]*(Uj[i+1] - Uj[i-1])
  end
  Ujp1[M] = Uj[M]
  push!(U, Ujp1)
end
```

[ ]:
```
graficos = []
for t=[1,11,51,101]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.5,1.6), label="U_ij", title="t=$tt")
  solucao = u.( domain .- u.(domain).*t )
  plot!(p, domain, solucao, label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Método Box

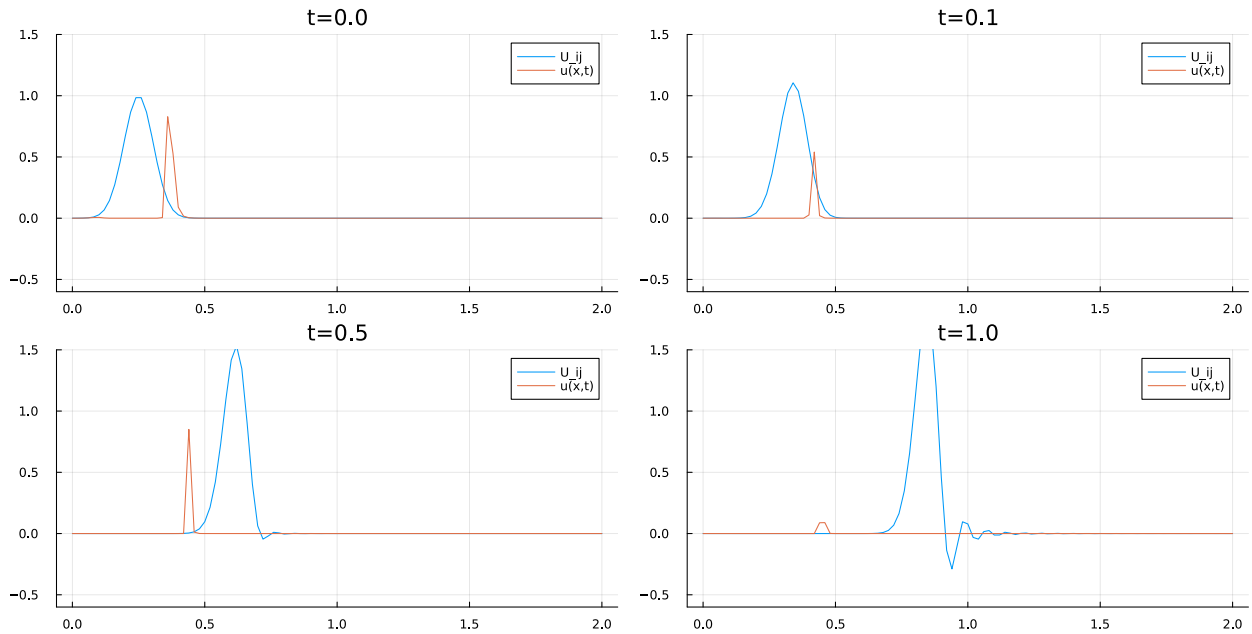
[ ]:
```
# Método Box
h = 0.02
k = 0.01

T = 100
```

```
domain = 0:h:2

U0 =  u.(domain);


U = [U0]
ν = k/h

M = length(U0)


for j=1:T
  Uj = U[j]
  diagA = ones(M)
  subdiagB = zeros(M)
  for i=2:M-1
    xi = i*h
    tj = j*k
    va = ν*a(xi,tj)
    λ = (1 + va)/(1 - va)
    diagA[i] = λ
    subdiagB[i] = λ
  end
  A = Tridiagonal(ones(M-1), diagA, zeros(M-1))
  B = Tridiagonal(subdiagB[1:M-1], ones(M), zeros(M-1))
  Ujp1 = A\(B*Uj)
  push!(U, Ujp1)
end
```

[ ]:

```
graficos = []
for t=[1,11,51,101]
  tt = (t-1)*k
  p = plot(domain, U[t], ylim=(-0.6,1.5), label="U_ij", title="t=$tt")
  solucao = u.( domain .- u.(domain).*t )
  plot!(p, domain, solucao, label="u(x,t)")
  push!(graficos, p)
end
plot(graficos...; layout=@layout([° °; ° °]), size=(1200,600))
```

[ ]:



Esse método até conseguiu lidar com a onda de choque, porém com efeitos de ganho de amplitude.

1.4 [5.40]

Derive um esquema explícito, usando diferenças centrais, para resolver

$$u_{xx} - (1-4x)^2 u_{tt} = 0, \quad 0 < x < 1, \quad t > 0,$$

com

$$u(x,0) = x^2, \quad u_t(x,0) = 0, \quad u_x(0,t) = 0, \quad u(1,t) = 1.$$

Mostre que as condições de fronteira estão incluídas no método numérico. **Estabeleça as condições CFL** para esse método.

1.4.1 Diferenças centradas

Usando uma aproximação de segunda ordem para u_{xx} e u_{tt} temos

$$u_{xx} = a^2(x) t_{tt}$$
$$\frac{U_{i+1,j} - 2U_{i,j} + U_{i-1,j}}{h^2} = a^2(x_i) \frac{U_{i,j+1} - 2U_{i,j} + U_{i,j-1}}{j^2}$$

$$U_{i,j+1} = 2U_{i,j} - U_{i,j-1} + \left(\frac{\lambda}{a(x_i)}\right)^2 \left(U_{i+1,j} - 2U_{i,j} + U_{i-1,j}\right), \qquad i = 1, \ldots, N-1$$

As condições de fronteira nos dão

- $$u_x(0,t) \approx \frac{U_{1,j} - U_{0,j}}{h} = 0 \implies U_{1,j} = U_{0,j}$$

- $$u(1,t) = 1 \implies U_{N,j} \equiv 1$$

1.4.2 Convergência do método

Realizando a mesma análise de convergência utilizada na questão 1, e chegamos na seguinte desigualdade

$$\frac{\lambda}{a(x_i)} \leq 1$$

Dessa forma, precisamos tomar o λ menor do que o mínimo entre os $a(x_i)$ para que o método convirja.

1.4.3 Resolvendo

[ ]:
```
a(x) = 1-4x # velocidade da onda
f(x) = x^2  # perfil inicial
```

[ ]:
```
f (generic function with 1 method)
```

[ ]:
```
# Vamos definir o domínio e plotar a curva inicial
N = 25*2
h = 1/(N-1)
domain = 0:h:1;
h
```

[ ]:
```
0.02040816326530612
```

Vamos usar $h \approx$ `6e-3`.

Vamos verificar se nosso ponto de descontinuidade não está na malha, pois não queremos de maneira alguma dividir por zero.

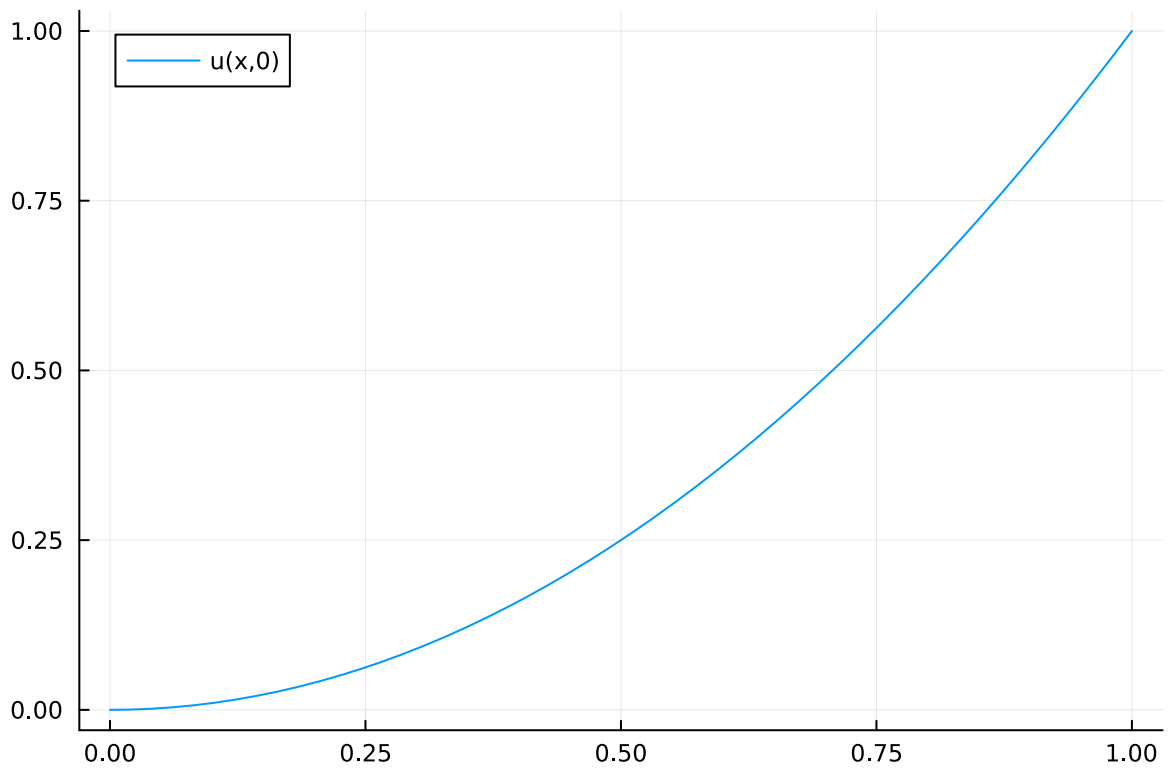
[ ]:
```
0.25 in domain
```

[ ]:
```
false
```

Vejamos o perfil inicial da nossa onda.

[ ]:
```
plot(domain, f.(domain), label="u(x,0)")
```

[ ]:



[ ]:
```
# LeapFrog
λ = min(abs.(a.(domain))...) # calculando o mínimo dos a(x_j)
k = λ*h
```

[ ]: 0.0004164931278633907

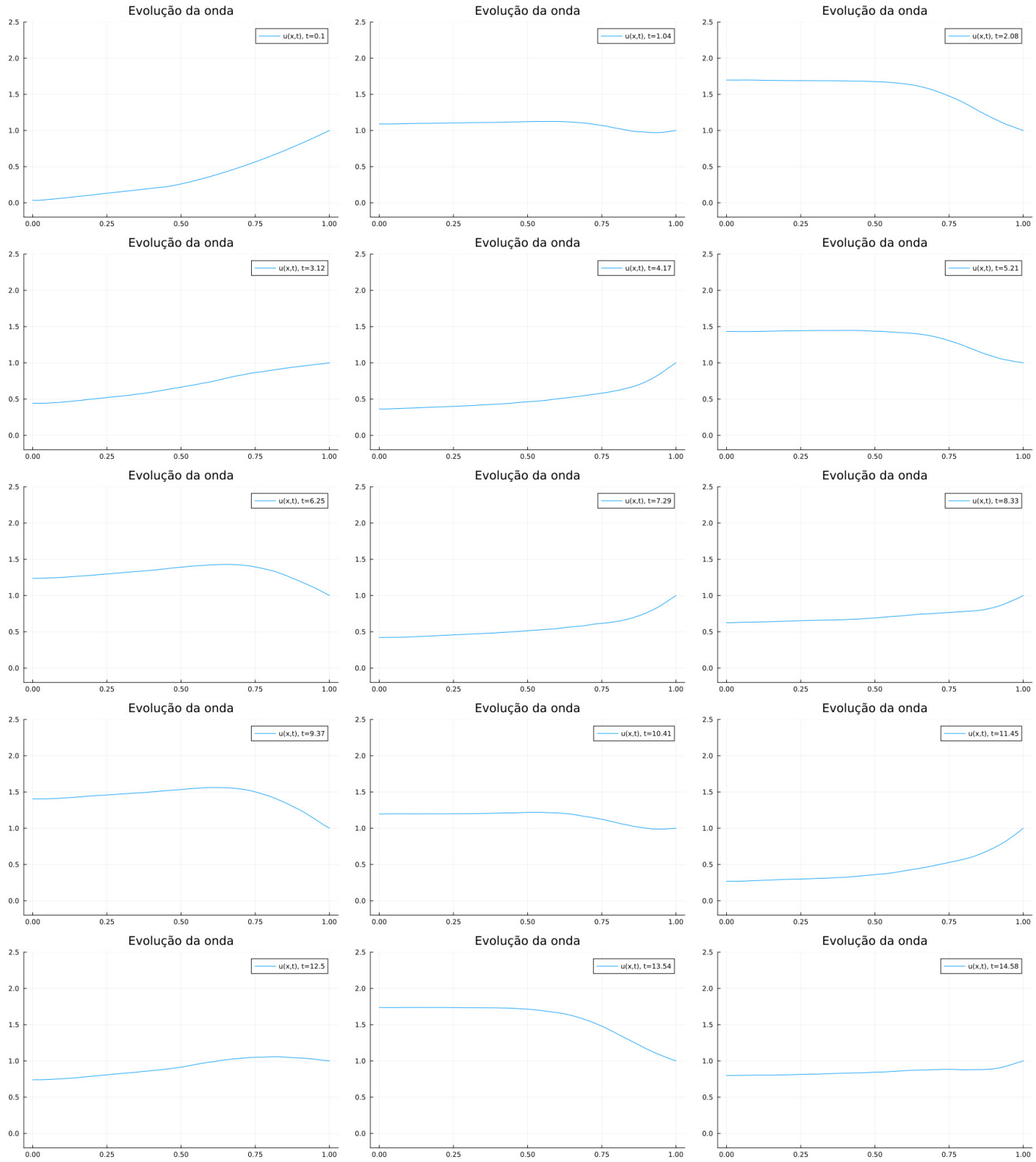
Como k é muito pequeno, temos que fazer uma simulação gigantesca para ter um intervalo de tempo bom.

[ ]:
```
T = 50000

T*k # Tf
```

[ ]: 20.824656393169537

[ ]:
```
U0 =  f.(domain);
U1 = copy(U0); # U^1 = U^0

U = [U0, U1]


M = length(U0)

for j=2:T
  Ujp1 = zeros(M) # Ut[0] = 0 por condjção jnjcjal
  Uj = U[j]
```

```
  Ujm1 = U[j-1]
  for i=2:M-1  # para os pontos interjores fazemos o esquema leapfrog
    Ujp1[i] = 2Uj[i] - Ujm1[i] + (λ/a(i*h))^2*(Uj[i+1]+Uj[i-1] - 2*Uj[i])
  end
  # e para o ponto da frontejra, usamos uma aproxjmação de 1a ordem
  Ujp1[M] = 1
  Ujp1[1] = Ujp1[2]
  push!(U, Ujp1)
end
```



1.5 Conclusões

Dentre os problemas apresentados, os mais difíceis são os que não tem domínio limitado, tornando difícil a implementação para os pontos extremos.

Outra parte complicada para os métodos numéricos é lidar com descontinuidades.

A parte difícil das equações de advecção são principalmente a formação de ondas de choque.